Big Data Frameworks

Batch Processing

Marvin Moughabghab

Section I

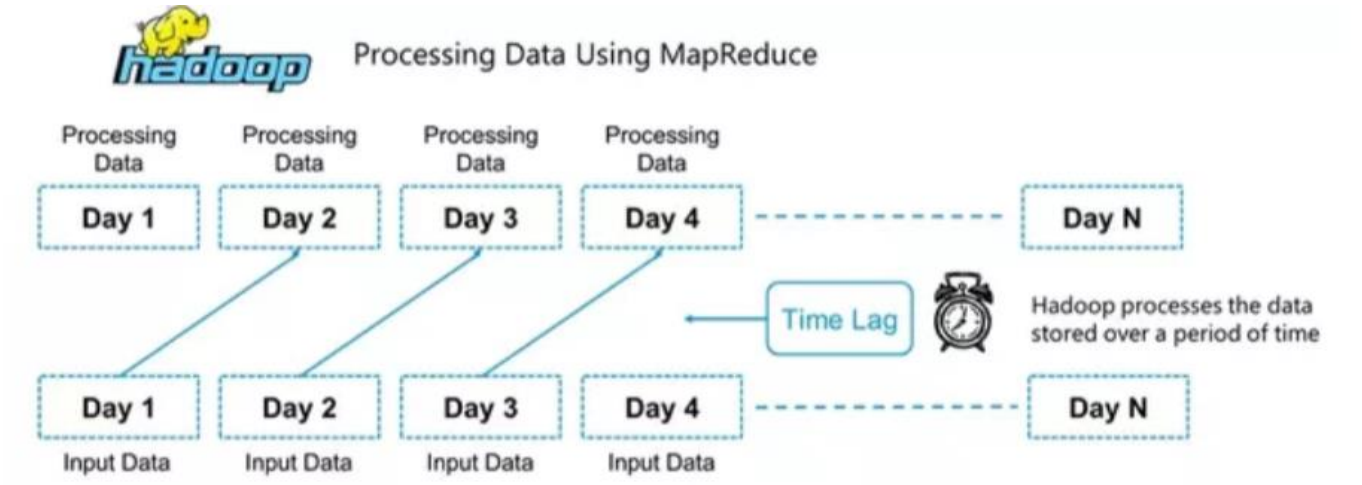
What is Batch Processing ?

What is Batch Processing ?

- ***Batch processing*** is when the processing happens on the **blocks of data** that have **already been stored** over a period of time.

- Batch processing is used mainly when we don't need real-time analytics results, and when it is more important to process large volumes of data to get more detailed insights than it is to get fast analytics results.

- For example, processing all the transaction that have been performed by a major financial firm in a week.


Processing Data Using MapReduce
Processing Data
Processing Data
Processing Data
Processing Data
Day 1
Day 2
Day 3
Day 4
Day N
Time Lag
Hadoop processes the data stored over a period of time
Day 1
Day 2
Day 3
Day 4
Day N
Input Data
Input Data
Input Data
Input Data

Batch Processing Frameworks

- Hadoop (MapReduce, HDFS, Yarn etc.)
- Apache Spark
- Apache Flink

Section II

What is Hadoop ?

What is Hadoop ?

- Hadoop is a software library and a framework that allows the distributed processing of large datasets across computer clusters using simple programming models

- Hadoop technology relies on **disk intensive** techniques:
  - Load -> Process -> Store

What is Hadoop ?

- Hadoop components:
  - Hadoop Distributed File System (HDFS)
  - Hadoop MapReduce
  - Hadoop Common → Collection of libs and utilities
  - Hadoop YARN (Yet Another Resources Negotiator)

- Hadoop technology relies on **disk intensive** techniques:
  - Load -> Process -> Store

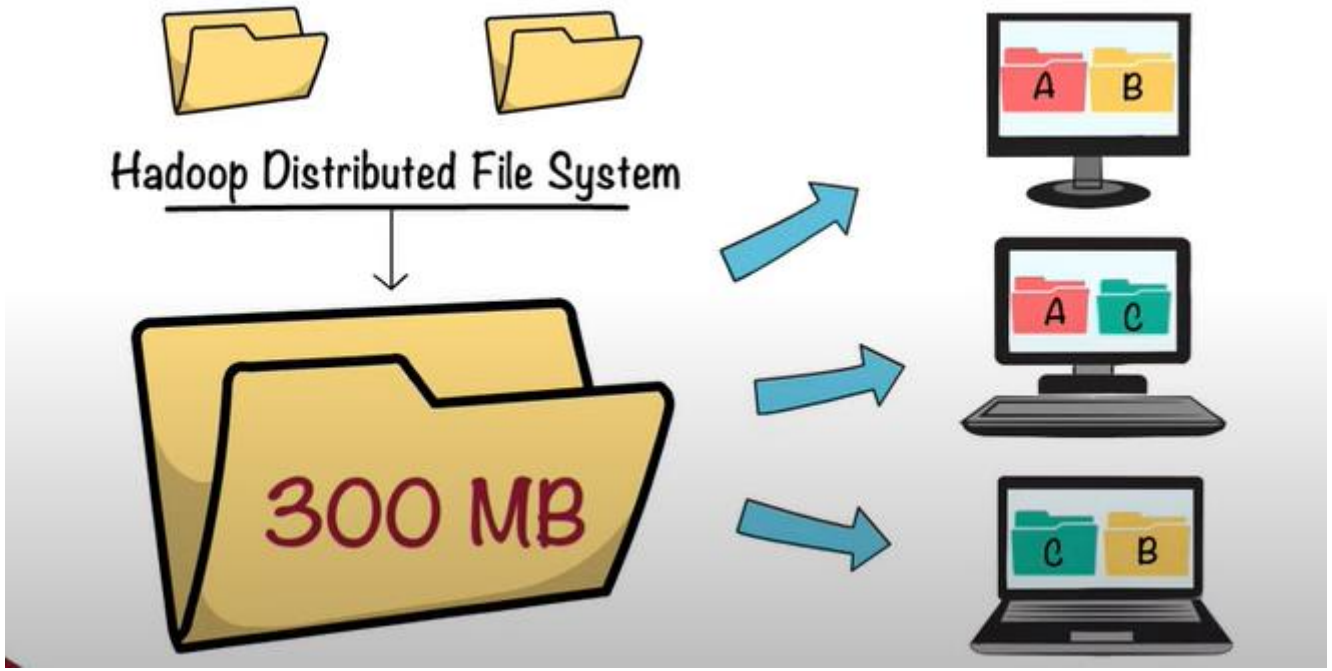
What is Hadoop ?

- Projects Based on Hadoop:
  - Cassandra
    - database for scalability and high availability without compromising performance
    - Support for replicating across multiple
  - Hive
    - Data warehouse software facilitates reading, writing, and managing large datasets residing in distributed storage using SQL
  - Pig
    - platform for analyzing large data sets that consists of a high-level language for expressing data analysis programs, coupled with infrastructure for evaluating these programs

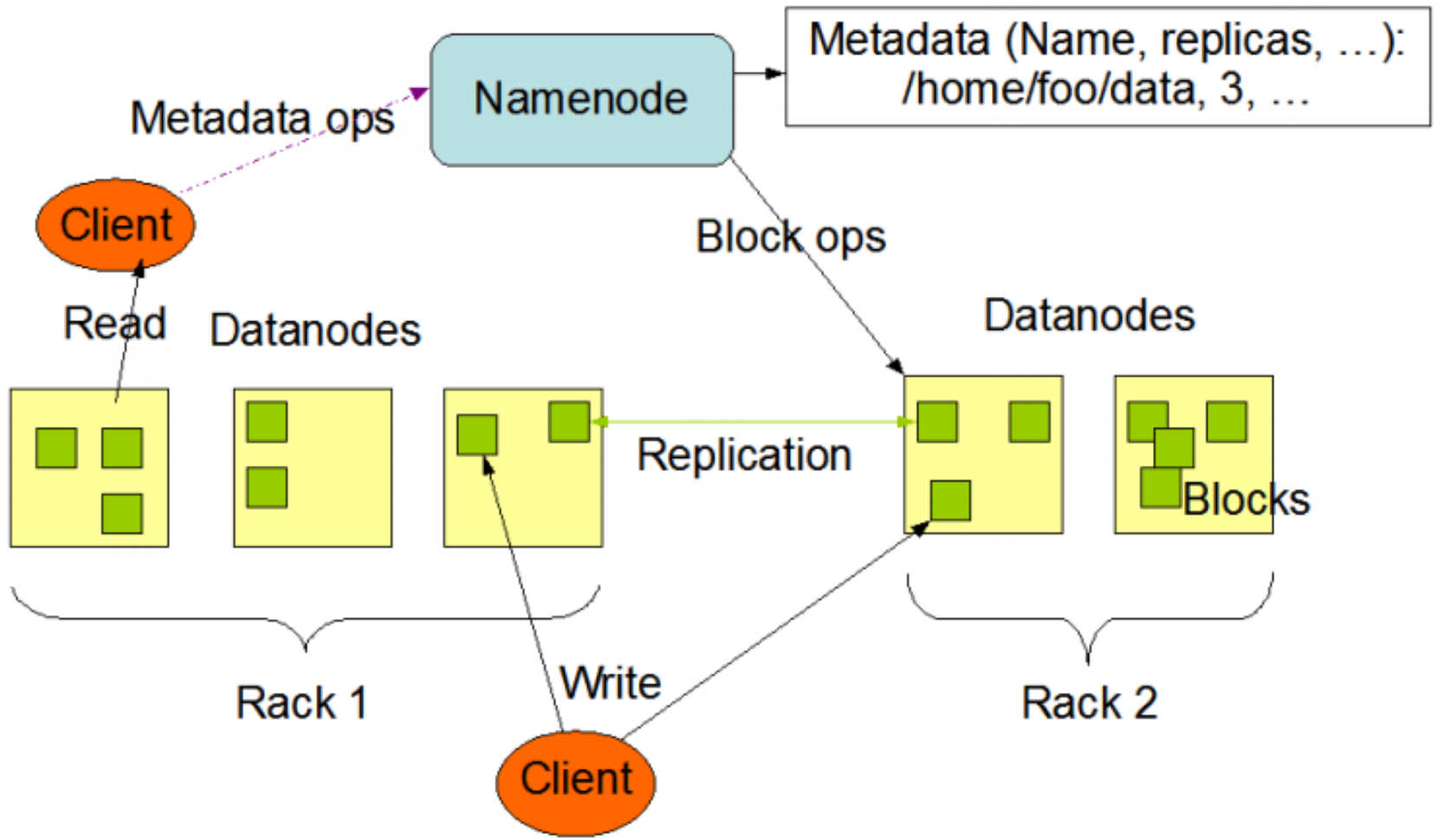
Section III

Hadoop Distributed File System (HDFS)

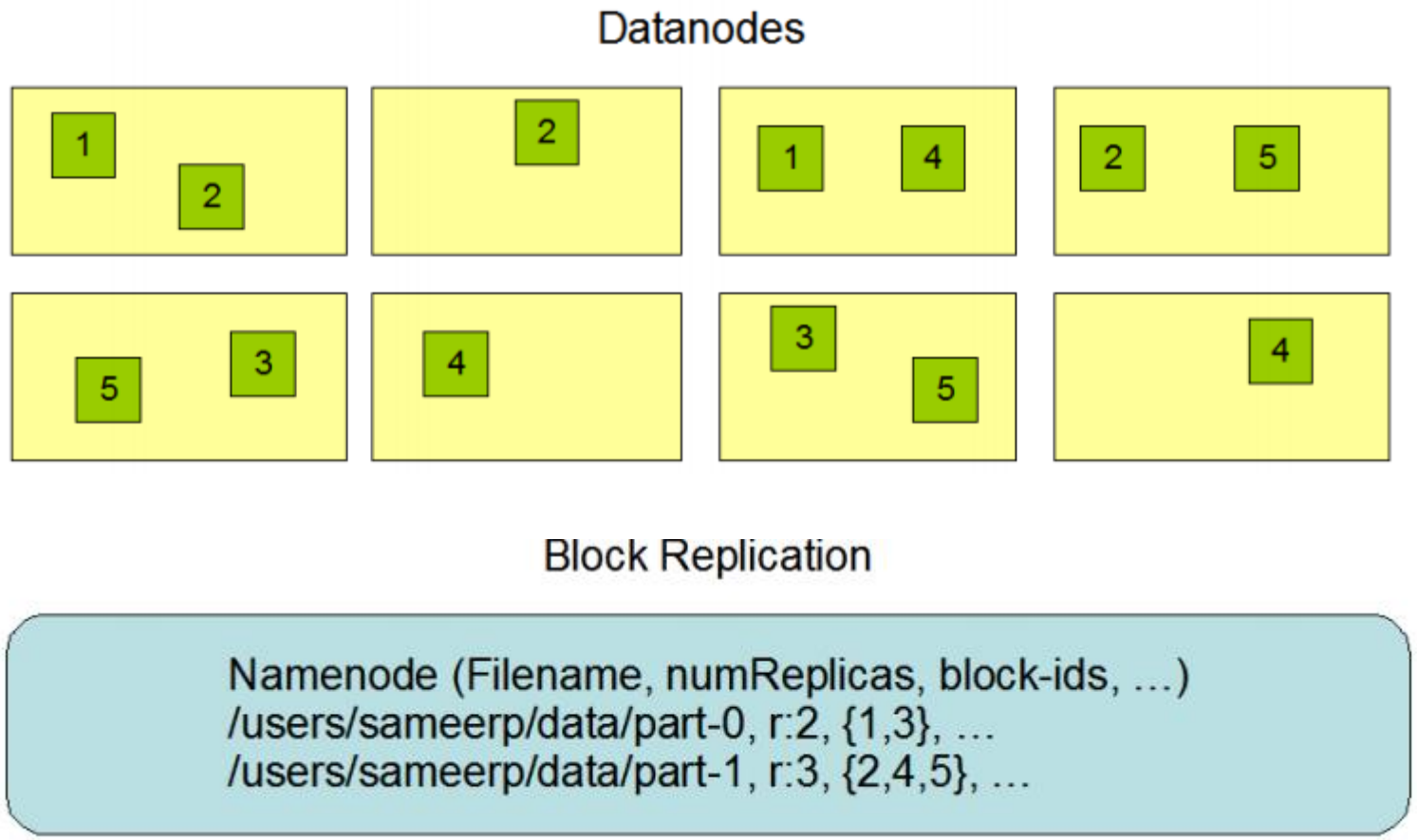
HDFS



HDFS Architecture



HDFS Architecture



HDFS Architecture

- HDFS relies on a Master/Slave architecture
- HDFS architecture is centered around **1 NameNode:**
  - master server that manages the file system namespace and regulates access to files by clients
  - executes file system namespace operations like opening, closing, and renaming files and directories
  - determines the mapping of blocks to DataNodes
  - Internally, a file is split into one or more blocks and these blocks are stored in a set of DataNodes

HDFS Architecture

- HDFS architecture relies on **multiple DataNodes:**
  - manage storage attached to the nodes that they run on
  - perform block creation, deletion, and replication **upon instruction from the NameNode**
  - responsible for serving read and write requests from the file (like in any other file system)

HDFS Architecture

- HDFS exposes a file system namespace and allows user data to be stored in files
- The NameNode is the arbitrator and repository for all HDFS metadata. The system is designed in such a way that user data never flows through the NameNode
- Each file is stored as a sequence of blocks
- All blocks are the same size except the last block
- Files are write-once (and one write at a time)

HDFS Features

- The NameNode periodically receives a Heartbeat and a Blockreport from each of the DataNodes in the cluster.
- Receipt of a Heartbeat implies that the DataNode is functioning properly.
- A Blockreport contains a list of all blocks on a DataNode.

HDFS Characteristics

I- Replication

- HDFS stores each file as a sequence of blocks
- The blocks of a file are replicated for fault tolerance
- Replication options can be modified at any time
- Replicas or Replication Factor is equal to 3 by default

II- Robustness

- Failures can occur to the nodes and network partitions which can lead to different problems:
  - Data Integrity: The HDFS client software implements checking on the contents of HDFS file. When a client creates an HDFS file, it computes a checksum of each block of the file and stores these checksums in a separate hidden file in the same HDFS namespace. This checksum is verified again in the namespace each time we retrieve the block's data
  - Cluster Rebalancing: A scheme automatically moves data from one DataNode to another **if the free space on a DataNode falls behind a certain threshold**
  - Data Disk Failure: Is treated with the concept of Heartbeats. **DataNodes** without heartbeats are considered as dead. Re-replication might be required.

HDFS Characteristics

III- Accessibility

- A command line interface called FS shell lets a user interact with the data in HDFS. The syntax is similar to other user-friendly shells (e.g: bash, csh)
- e.g: Create a directory named /foodir → **bin/hadoop dfs -mkdir /foodir**

IV - Data Organization

- A typical block size used by HDFS is 64 MB
- New created files, in reality, do not reach the NameNode directly:
  - HDFS will caches the file data into a temporary local file until the local file accumulates data worth over HDFS block size →Better performance

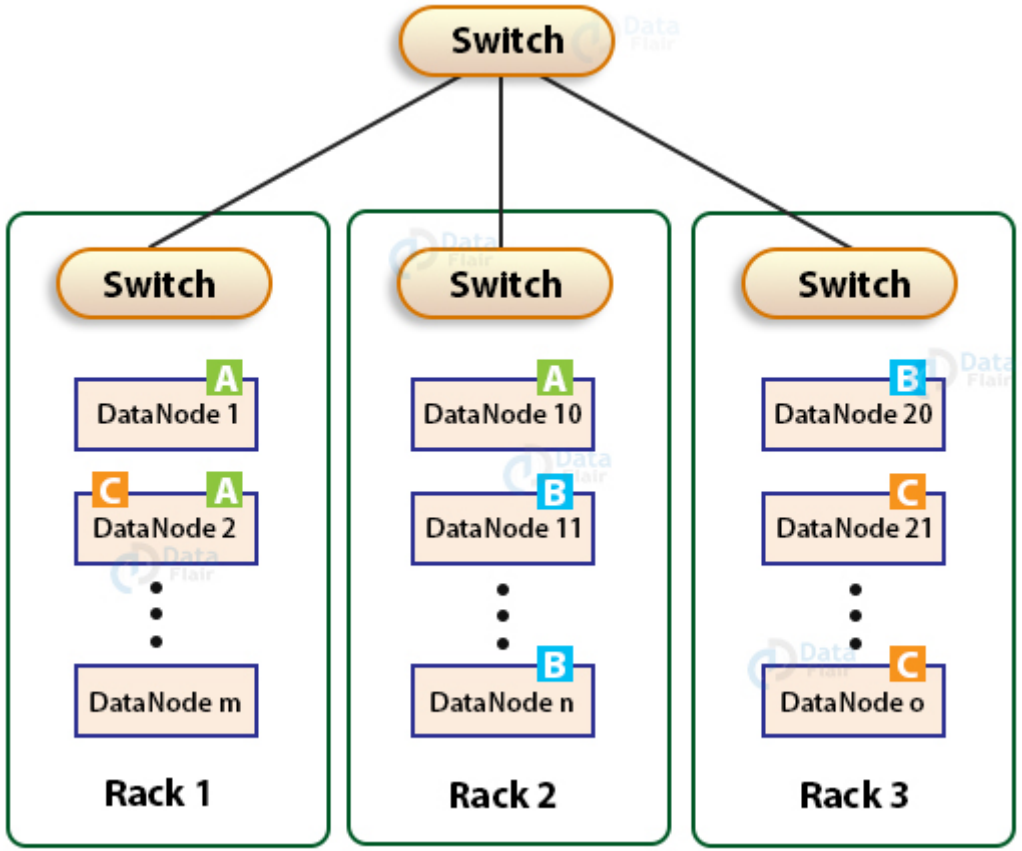
V – Persistence

- The NameNode uses a transaction log called the EditLog to persistently record every change that occurs to file system metadata.
- The entire file system namespace, including the mapping of blocks to files and file system properties, is stored in a file called the FsImage.
- Reloading the EditLog transactions and FsImage is called checkpointing (At startup or periodically)

Hadoop Rack Awareness

- **Rack Awareness** enables Hadoop to maximize network bandwidth by favoring the transfer of blocks within **racks** over transfer between **racks**. Especially with **rack awareness**, the YARN is able to optimize MapReduce job performance. It assigns tasks to nodes that are 'closer' to their data in terms of network topology.

Hadoop Rack Awareness



Hadoop Rack Awareness

- The purpose of a rack-aware replica placement policy is to improve data reliability, availability, and network bandwidth utilization
- Communication between two nodes in different racks has to go through switches
- In most cases, network bandwidth between machines in the same rack is greater than network bandwidth between machines in different racks (allows rack local tasks)
- The NameNode determines the rack id each DataNode belongs

Hadoop Rack Awareness Policies

- A simple but non-optimal policy is to place replicas on unique racks (replicas distributed across racks):
  - This prevents losing data when an entire rack fails and allows use of bandwidth from multiple racks when reading data
  - This policy evenly distributes replicas in the cluster which makes it easy to balance load on component failure
  - However, this policy increases the cost of writes because a write needs to transfer blocks to multiple racks

Hadoop Rack Awareness Policies

- When the replication factor is three, HDFS's placement policy is to put one replica on one node in a first rack, another on a node in a different second rack, and the last on a different node in the same second rack:
  - This policy cuts the inter-rack write traffic which generally improves write performance without compromising data reliability or read performance
  - The chance of rack failure is far less than that of node failure
  - However, it does reduce the aggregate network bandwidth used when reading data since a block is placed in only two unique racks rather than three

Hadoop Rack Awareness

- Rack Local Tasks